

EcomFlow

Spécification fonctionnelle, technique et produit — Version 2.0

Plateforme SaaS de gestion et d'automatisation des opérations e-commerce en Algérie

DOCUMENT	VERSION	DATE
Cahier des charges complet	2.0	29 août 2026
MODÈLE	ESSAI	CIBLE
SaaS multi-tenant	7 jours gratuits	E-commerçants algériens

OBJECTIF : fournir une spécification suffisamment précise pour permettre à une équipe de développement ou à un système d'IA de concevoir et implémenter le produit sans inventer les règles métier essentielles.

1. Executive Summary

EcomFlow est un SaaS destiné aux e-commerçants algériens. Il centralise les commandes issues de Google Sheets et, à terme, d'autres canaux, puis orchestre leur confirmation, préparation, expédition, tracking, livraison, retour et analyse.

Le produit doit être pensé comme une plateforme d'opérations e-commerce et non comme un simple tableau de commandes. Son cœur est un moteur de workflow et d'intégrations.

Le parcours cible est : Google Sheets → validation → confirmation téléphonique → préparation → transporteur → tracking → livraison/retour → reporting.

- Positionnement proche d'un Ecomanager avec une couche d'automatisation et d'intégration plus structurée.
- Architecture extensible pour plusieurs boutiques, utilisateurs et transporteurs.

2. Objectifs et KPI produit

- Réduire au minimum la ressaisie manuelle.
- Éviter les doublons et incohérences de commandes.
- Réduire le temps entre confirmation et expédition.
- Centraliser le suivi transporteur.
- Permettre à un manager de mesurer confirmation, livraison, retours et rentabilité.
- Transformer le service en abonnement SaaS après un essai gratuit de 7 jours.

3. Périmètre

- Accounts & authentication
- Boutiques / tenants
- Utilisateurs, rôles et permissions
- Commandes
- Clients
- Produits, variantes et stock
- Google Sheets
- Centre de confirmation
- Expéditions et transporteurs
- Tracking
- Retours
- Dashboard & reporting
- Notifications
- Audit logs
- Abonnements et paiements
- Paramètres
- Import/export

4. Utilisateurs et permissions

Le système doit fonctionner avec des permissions par rôle et, si nécessaire, des permissions granulaires. Toute requête backend doit vérifier le tenant et les droits avant d'accéder aux données.

- SUPER_ADMIN : plateforme, tenants, plans, incidents et configuration globale.
- OWNER/ADMIN : boutique, utilisateurs, commandes, intégrations, stock, statistiques et abonnement.
- MANAGER : opérations et reporting autorisés.
- CONFIRMATION_AGENT : file de confirmation, appels, notes et statuts autorisés.
- PREPARER : préparation, packing et commandes prêtes à expédier.

5. Multi-tenant & isolation des données

Chaque boutique est un tenant logique. Toute entité métier doit être rattachée au `tenant_id`. Les contrôles d'accès doivent empêcher qu'un utilisateur consulte ou modifie les données d'une autre boutique.

Prévoir une stratégie d'isolation compatible avec la croissance : index `tenant_id`, filtrage systématique côté service/repository et tests automatisés d'isolation.

6. Authentification et onboarding

- Inscription email + mot de passe avec validation.
- Connexion, déconnexion, récupération de mot de passe et gestion de session.
- OAuth Google pour l'intégration Google Sheets, avec consentement explicite.
- Après inscription : création de la boutique, configuration minimale, démarrage du Trial de 7 jours et accès au dashboard.
- Afficher immédiatement `trial_start_at`, `trial_end_at` et le nombre de jours restants.

7. Essai gratuit et abonnement

Chaque nouvelle boutique bénéficie d'un essai gratuit de 7 jours, sans renouvellement automatique du Trial. À l'issue des 7 jours, un abonnement actif est requis.

Le système doit avoir une source de vérité côté serveur pour la période d'essai et l'état de l'abonnement. L'interface ne doit jamais décider seule qu'un compte est actif.

- `TRIAL_ACTIVE` : fonctionnalités disponibles pendant 7 jours.
- `TRIAL_ENDING` : état d'interface indiquant une expiration proche.
- `TRIAL_ENDED/EXPIRED` : expiration sans abonnement actif ; fonctionnalités opérationnelles suspendues.
- `ACTIVE` : abonnement valide.
- `PAST_DUE` : paiement à régulariser selon le prestataire.
- `SUSPENDED` : accès opérationnel suspendu.
- `CANCELLED` : abonnement résilié.
- Conserver les données après expiration pendant une durée configurable.

8. Plans tarifaires

Le moteur d'abonnement doit être configurable et indépendant du frontend. Les plans peuvent définir prix, période, nombre de commandes, utilisateurs, boutiques, intégrations et fonctionnalités.

Prévoir Starter, Pro et Business comme structure initiale, sans figer les prix dans le code.

Le paiement doit être confirmé par le mécanisme du prestataire retenu. Une simple preuve visuelle ne doit pas déclencher automatiquement l'activation.

9. Commandes — modèle fonctionnel

Une commande reçoit un ID interne unique du type `ORD-AAAA-XXXXXX` et peut conserver un `external_order_id` provenant de Google Sheets ou d'une autre source.

- Informations client : nom, téléphone, adresse, wilaya, commune.
- Commande : produits, variantes, quantités, prix unitaires, remise éventuelle, frais de livraison, total.
- Opérations : source, statut, notes, agent, dates métier.
- Livraison : transporteur, tracking, service, statut transporteur, événements.
- Retour : statut, motif, date, état produit, décision stock.

10. Workflow et règles de statut

Workflow nominal : `NEW` → `À CONFIRMER` → `CONFIRMÉE` → `EN PRÉPARATION` → `PRÊTE À EXPÉDIER` → `EXPÉDIÉE` → `EN LIVRAISON` → `LIVRÉE`.

Workflow alternatif : `NO ANSWER`, `CALL BACK`, `POSTPONED`, `CANCELLED`, `REFUSED`, `RETURNED`, `NUMÉRO INCORRECT`.

Les transitions doivent être définies dans un service de workflow et contrôlées côté backend. Une transition invalide doit retourner une erreur métier explicite.

Chaque transition conserve : actor_id, timestamp, old_status, new_status, reason, note et source.

11. Centre de confirmation téléphonique

- File de travail triée par priorité et date.
- Fiche commande avec informations client et produit.
- Actions rapides : confirmer, rappeler, reporter, annuler, numéro incorrect.
- Nombre et historique des tentatives de contact.
- Date/heure du prochain rappel pour les commandes CALL BACK/POSTPONED.
- Notes internes et agent responsable.
- Filtres par statut, date, agent, wilaya, source et priorité.

12. Google Sheets — spécification d'intégration

Le connecteur Google Sheets doit utiliser OAuth et stocker uniquement les tokens nécessaires, de manière sécurisée. Une boutique peut sélectionner un Spreadsheet et une Sheet.

Le mapping des colonnes doit être configurable afin de ne pas imposer un format unique.

Le système doit traiter les nouvelles lignes, valider les données, générer une clé d'idempotence et journaliser le résultat.

- Mapping recommandé : Date, Nom, Téléphone, Wilaya, Commune, Adresse, Produit, SKU, Quantité, Prix, Livraison, Total, Statut.
- Normalisation : téléphone, espaces, formats numériques et valeurs de wilaya/commune.
- Validation : champs obligatoires, quantité positive, prix cohérent, téléphone exploitable.
- Déduplication : external_row_id ou hash stable des champs source ; jamais de double création lors d'une resynchronisation.
- Gestion des erreurs : ligne en erreur, raison, date, tentative et possibilité de retry.
- Synchronisation périodique + bouton Sync Now.
- Conserver l'origine et l'identifiant de la ligne source.

13. Gestion des clients

- Créer ou retrouver un client à partir d'un identifiant métier principal, notamment téléphone, sans fusion automatique risquée.
- Historique des commandes.
- Nombre de commandes, livrées, annulées et retournées.
- Adresse(s) et informations utiles avec contrôle des doublons.
- Notes internes et éventuellement tags.
- Afficher un historique client dans la fiche commande.

14. Produits, variantes et stock

Le catalogue doit gérer produits, SKU, variantes et mouvements de stock. Les règles de décrémentation doivent être centralisées afin d'éviter des écarts entre modules.

- Produit : nom, SKU, catégorie, prix achat, prix vente, image, description, actif/inactif.
- Variante : SKU, couleur, taille ou attributs, stock propre.
- Mouvements : type, quantité, référence, commande, utilisateur, date et note.
- Types : entrée, réservation, sortie, annulation, retour, ajustement.
- Seuil de stock configurable et alerte.
- Retour : réintégration au stock disponible ou stock à vérifier.

15. Préparation et expédition

Les commandes CONFIRMÉES passent dans une file de préparation. L'utilisateur peut les préparer, vérifier les quantités puis les faire passer à PRÊTE À EXPÉDIER.

Avant création du colis, vérifier les données obligatoires et empêcher l'envoi si une condition bloquante n'est pas satisfaite.

16. Architecture des intégrations transporteurs

Créer une interface de connecteur commune afin d'intégrer plusieurs transporteurs sans modifier les services métiers principaux.

Exemple d'adaptateurs possibles selon disponibilité et documentation des APIs : Yalidine, ZR Express, Ecotrack et autres.

- createShipment(order)
- cancelShipment(shipment)
- getShipmentStatus(tracking)
- getTrackingEvents(tracking)
- normalizeStatus(providerStatus)
- healthCheck()
- Gestion des erreurs, timeout, retry avec backoff et idempotency key.
- Les secrets d'API sont chiffrés/ protégés et masqués dans les logs.

17. Création du colis et tracking

Quand la commande est éligible : choisir transporteur → envoyer la commande → recevoir tracking → enregistrer shipment → afficher tracking dans la commande.

Le système doit éviter de créer deux colis pour la même commande par erreur.

Les événements du transporteur sont normalisés vers les statuts internes EcomFlow.

- Exemple : EXPÉDIÉE → CENTRE DE TRI → ARRIVÉE À DESTINATION → EN LIVRAISON → LIVRÉE.
- Utiliser webhook si disponible ; sinon synchronisation périodique.
- Conserver provider_status et normalized_status pour diagnostic et reporting.

18. Retours, refus et anomalies

- Créer un retour lié à la commande originale.
- Motifs configurables : absent, refus, numéro incorrect, produit non conforme, injoignable, autre.
- Conserver les événements transporteur et les notes internes.
- Déterminer si le produit réintègre le stock ou passe en stock à vérifier.
- Les retours doivent apparaître dans les KPIs et rapports.

19. Détection des doublons

Le système doit détecter les commandes potentiellement identiques avant création ou confirmation. Les règles peuvent combiner téléphone, produit, adresse et fenêtre temporelle.

Une alerte ne doit pas supprimer automatiquement une commande. La décision finale appartient à un utilisateur autorisé.

20. Dashboard et KPIs

- Commandes totales, nouvelles, à confirmer, confirmées, expédiées, en livraison, livrées, retours, annulations.
- Taux de confirmation = commandes confirmées / commandes traitées selon définition métier.
- Taux de livraison = commandes livrées / commandes expédiées selon définition métier.
- Taux de retour = retours / commandes expédiées selon définition métier.
- Chiffre d'affaires et bénéfice estimé selon prix d'achat et coûts disponibles.
- Graphiques par jour, produit, wilaya, source, agent et transporteur.

- Filtres globaux par période et boutique.

21. Notifications et centre d'incidents

- Nouvelle commande.
- Stock faible.
- Commande livrée.
- Commande retournée.
- Échec Google Sheets.
- Échec transporteur/API.
- Expiration prochaine de l'essai.
- Échec ou problème d'abonnement.
- Les erreurs techniques doivent avoir un identifiant d'incident, un niveau, une date et une information exploitable pour le support.

22. Recherche, filtres et vues

- Recherche par ID, nom, téléphone, tracking, SKU ou produit.
- Filtres combinables par statut, période, wilaya, source, agent et transporteur.
- Pagination côté serveur.
- Tri configurable.
- Vues enregistrables dans une future version pour les équipes.

23. Audit log

Les opérations sensibles doivent être enregistrées : connexion, changement de rôle, modification de commande, changement de statut, intégration, export, activation/suspension d'abonnement.

Chaque log contient au minimum actor, tenant, action, entity, entity_id, timestamp et metadata non sensible.

24. Import / Export

- Import CSV/Excel avec mapping et validation.
- Export CSV/Excel/PDF des commandes, produits, retours et rapports.
- Respecter les permissions d'export.
- Les gros exports doivent être traités en tâche asynchrone dans une version avancée.

25. Pages et navigation UI

- Landing / présentation.
- Login / Register / Forgot Password.
- Onboarding boutique.
- Dashboard.
- Commandes.
- Centre de confirmation.
- Clients.
- Produits.
- Stock.
- Préparation.
- Expéditions.
- Tracking.
- Retours.
- Statistiques.
- Intégrations.
- Utilisateurs & rôles.

- Abonnement & facturation.
- Notifications / incidents.
- Paramètres.
- Administration Super Admin.

26. Architecture logicielle recommandée

Frontend : Next.js + React + TypeScript. Backend : NestJS + TypeScript. Database : PostgreSQL. Cache/queues : Redis si nécessaire. Automation : n8n pour les workflows externes, sans déplacer les règles métier critiques hors du backend.

API : REST documentée via OpenAPI/Swagger ; webhooks sécurisés pour les intégrations. Déploiement conteneurisé avec Docker.

- Séparer modules : Auth, Tenants, Users, Orders, Customers, Catalog, Inventory, Confirmation, Shipments, Returns, Integrations, Billing, Notifications, Audit.
- Utiliser services/repositories et validation DTO.
- Centraliser les règles métier et transitions de statut.
- Prévoir background jobs pour synchronisation, notifications, exports et retry API.

27. Base de données — entités principales

- tenants : id, name, slug, status, created_at, updated_at.
- users : id, email, password_hash/identity_ref, status, created_at, updated_at.
- memberships : id, tenant_id, user_id, role_id, status.
- roles / permissions / role_permissions.
- customers : id, tenant_id, name, phone_normalized, notes, created_at, updated_at.
- addresses : id, tenant_id, customer_id, wilaya, commune, address_text.
- products : id, tenant_id, name, sku, category_id, purchase_price, sale_price, status.
- product_variants : id, product_id, sku, attributes_json, stock settings.
- inventory_movements : id, tenant_id, product_variant_id, type, quantity, reference_type, reference_id, actor_id, created_at.
- orders : id, tenant_id, external_order_id, customer_id, source, status, totals, dates, assigned_user_id.
- order_items : id, order_id, product_variant_id, quantity, unit_price.
- order_status_history : id, order_id, old_status, new_status, reason, note, actor_id, created_at.
- shipments : id, order_id, provider_id, tracking_number, provider_status, normalized_status, status, created_at.
- shipment_events : id, shipment_id, provider_status, normalized_status, event_at, payload_ref.
- returns : id, order_id, reason, status, product_condition, stock_decision.
- integrations : id, tenant_id, type, provider, credentials_ref, status, config_json.
- subscriptions : id, tenant_id, plan_id, status, trial_start_at, trial_end_at, current_period_start, current_period_end.
- plans : id, name, price, billing_period, limits_json, features_json.
- payments : id, tenant_id, subscription_id, provider, external_payment_id, amount, status, paid_at.
- notifications / audit_logs / import_jobs / sync_runs.

28. API — familles d'endpoints

- POST /auth/register, POST /auth/login, POST /auth/refresh.
- GET/POST /tenants, GET/PATCH /tenants/:id.
- GET/POST /orders, GET/PATCH /orders/:id, POST /orders/:id/status.
- GET/POST /customers.
- GET/POST/PATCH /products, /variants, /inventory.
- GET /confirmation/queue, POST /orders/:id/calls, POST /orders/:id/confirm.
- GET/POST /shipments, POST /orders/:id/ship.
- GET /shipments/:id/tracking.

- POST /webhooks/:provider.
- GET/POST /integrations/google-sheets.
- GET/POST /subscriptions, GET /plans, POST /payments/webhook.
- GET /dashboard/kpis, GET /reports.
- GET /audit-logs.

29. Google Sheets — flux technique

1) OAuth Google. 2) Sélection Spreadsheet/Sheet. 3) Mapping colonnes. 4) Test de connexion. 5) Première synchronisation contrôlée. 6) Activation du job périodique. 7) Import des nouvelles lignes. 8) Validation. 9) Déduplication. 10) Création commande. 11) Journal de synchronisation.

Une synchronisation doit être idempotente. En cas d'échec réseau, elle doit pouvoir reprendre sans recréer les commandes déjà traitées.

30. Webhooks et jobs

- Webhook entrant : vérifier signature/token, timestamp éventuel et idempotency.
- Job de synchronisation Google Sheets.
- Job de tracking transporteurs.
- Job de retry des intégrations.
- Job de notifications.
- Job d'expiration des Trials et de vérification des abonnements.
- Les jobs doivent être observables avec statut, tentatives, erreur et durée.

31. Sécurité et conformité opérationnelle

- HTTPS obligatoire en production.
- Hashage robuste des mots de passe.
- Tokens OAuth/API protégés et non exposés au client.
- Validation côté serveur.
- Rate limiting et protection brute-force.
- Contrôle tenant + permissions sur chaque endpoint.
- Logs sans mots de passe, tokens ou données sensibles inutiles.
- Sauvegardes et procédure de restauration testée.
- Politique de conservation des données configurable.

32. Observabilité et maintenance

- Logs structurés.
- Health checks des services.
- Monitoring des erreurs API.
- Monitoring des jobs et files.
- Alertes sur intégrations indisponibles.
- Versionnage des migrations DB.
- Environnements séparés : development, staging, production.

33. Tests et qualité

- Unit tests : règles de statut, calculs, déduplication, stock.
- Integration tests : Google Sheets, transporteur mocké, abonnement.
- API tests : auth, permissions, isolation tenant.
- E2E : nouvelle commande → confirmation → préparation → expédition → tracking → livraison/retour.
- Tests spécifiques de Trial : création → J+7 → expiration → paiement → réactivation.
- Tests de non-régression avant chaque release.

34. Déploiement

- Application conteneurisée avec Docker.
- Base PostgreSQL avec migrations.
- Variables d'environnement pour secrets et URLs.
- HTTPS avec domaine.
- Backups DB automatisés.
- Stratégie de rollback.
- CI/CD recommandée.
- Stockage séparé pour médias et documents.

35. MVP — périmètre de lancement

- Auth + onboarding.
- Tenant/boutique.
- Trial 7 jours.
- Dashboard.
- Google Sheets import automatique.
- Commandes + workflow.
- Centre de confirmation.
- Produits + stock basique.
- Un connecteur transporteur.
- Création colis + tracking.
- Retours.
- Utilisateurs/permissions de base.
- Abonnement avec état ACTIVE/EXPIRED.

36. Roadmap

Phase 1 — MVP : cœur opérationnel et premier transporteur.

Phase 2 — Scale : plusieurs transporteurs, amélioration des intégrations, notifications et reporting avancé.

Phase 3 — Omnicanal : site web, réseaux sociaux et autres sources selon APIs disponibles.

Phase 4 — Mobile : application Flutter.

Phase 5 — Intelligence : aide à la priorisation, anomalies et prévisions, après validation des données et du ROI.

37. Critères d'acceptation finaux

- Une nouvelle ligne Google Sheets valide crée une seule commande.
- Une resynchronisation ne crée aucun doublon.
- Un agent peut confirmer, rappeler, reporter ou annuler selon ses permissions.
- Une commande confirmée peut suivre tout le workflow prévu.
- Une commande éligible peut générer un shipment via le connecteur.
- Le tracking est enregistré et les statuts sont normalisés.
- Les retours sont liés à la commande d'origine.
- Le stock respecte les mouvements et règles définis.
- Les données sont isolées par tenant.
- Les événements critiques sont historisés.
- Le Trial dure exactement 7 jours selon la règle temporelle définie côté serveur.
- Après expiration sans abonnement actif, les fonctionnalités opérationnelles sont bloquées.
- Après paiement confirmé, le compte redevient actif.
- Les erreurs d'intégration sont visibles et rejouables.

38. Règles de conception à ne pas violer

- Ne pas coder les prix des plans directement dans le frontend.
- Ne pas faire confiance au frontend pour les permissions ou le statut d'abonnement.
- Ne pas créer de commande sans mécanisme d'idempotence pour les intégrations.
- Ne pas coupler les règles métier à un transporteur unique.
- Ne pas stocker de secrets dans le code source.
- Ne pas mélanger les données entre tenants.
- Ne pas supprimer l'historique métier important lors d'un changement de statut.
- Ne pas utiliser n8n comme unique source de vérité pour les commandes ou paiements : le backend EcomFlow reste la source métier.

39. Exemple de scénario complet

Une commande est ajoutée dans Google Sheets. Le job détecte la nouvelle ligne, valide les données et crée ORD-2026-000001. Elle apparaît dans la file À CONFIRMER. L'agent l'ouvre, contacte le client et choisit CONFIRMÉE. EcomFlow vérifie le stock et crée la tâche de préparation. Après préparation, la commande devient PRÊTE À EXPÉDIER. L'utilisateur choisit un transporteur compatible. L'API crée le colis et renvoie un tracking. EcomFlow enregistre le shipment et synchronise ensuite les événements. Lorsque le colis est livré, la commande passe à LIVRÉE et les KPI sont mis à jour. Si le transporteur retourne le colis, EcomFlow crée le retour et applique la règle de stock configurée.

40. Annexe — structure Google Sheets recommandée

Les noms de colonnes doivent rester configurables ; cette structure constitue le format de référence du MVP.

Date	Nom	Téléphon e	Wilay a	Commu ne	Adresse	Produit	SKU	Qt é	Prix	Livra ison	Tota l	Source
29/08/ 2026	Sara	05XXXXX XXX	Alger	Bab Ezzouar	Adresse client	Robe	ROB-0 01	1	450 0	500	500 0	Google Sheets

41. Annexe — états du compte

- TRIAL_ACTIVE : essai gratuit actif.
- TRIAL_ENDING : indicateur UI, sans modifier les droits seul.
- TRIAL_ENDED / EXPIRED : période terminée sans abonnement.
- ACTIVE : abonnement valide.
- PAST_DUE : paiement à régulariser.
- SUSPENDED : accès opérationnel suspendu.
- CANCELLED : abonnement résilié.

42. Note de cadrage

Les noms et exemples de transporteurs sont des cibles d'intégration et ne constituent pas une garantie de disponibilité d'une API. Les prix des abonnements, les moyens de paiement et certaines règles de stock doivent être configurables. Les détails contractuels et réglementaires devront être validés avant la mise en production.